

Usare Wireshark per Osservare l'Handshake a 3 Vie TCP

Laboratorio Cisco CyberOps: cattura e analisi della sessione TCP

Autore: BlackUtopia | Ambiente: VM CyberOps Workstation (Mininet) | Data: 16/06/2026

1. Obiettivo

In questo laboratorio ho catturato e analizzato il traffico generato durante l'apertura di una sessione TCP tra un client e un server web, con l'obiettivo di osservare nel dettaglio l'handshake a tre vie. L'handshake e' il meccanismo con cui due host stabiliscono una connessione affidabile prima di scambiare dati applicativi: lo studio dei suoi tre segmenti (SYN, SYN-ACK, ACK) permette di capire come vengono negoziati i numeri di sequenza, le porte e i parametri della connessione. Ho svolto l'attivita' in tre fasi: ho preparato gli host e catturato il traffico con tcpdump, ho analizzato i pacchetti con Wireshark applicando un filtro di visualizzazione, e infine ho riletto la cattura offline con tcpdump, consultando il manuale per comprendere l'opzione di lettura da file.

2. Configurazione dell'Ambiente

Ho lavorato all'interno della macchina virtuale CyberOps Workstation, che ospita una topologia di rete emulata con Mininet. Mininet crea host, switch e router virtuali collegati tra loro, in modo da riprodurre una piccola rete reale senza hardware fisico. Di seguito i parametri principali dell'ambiente.

Macchina virtuale	CyberOps Workstation, utente analyst
Emulatore di rete	Mininet, avviato con lo script cyberops_topo.py
Topologia	Router R1, switch S1, host H1-H4
Rete client	10.0.0.0/24, attestata su R1-eth1
Rete server	172.16.0.0/12, attestata su R1-eth2
Host client	H1, indirizzo 10.0.0.11
Server web	H4, indirizzo 172.16.0.40, servizio nginx
Strumenti	tcpdump, Wireshark 4.4.7, Firefox
File di cattura	/home/analyst/capture.pcap

3. Parte 1: Preparazione degli Host e Cattura del Traffico

3.1 Avvio della topologia Mininet

Ho avviato la topologia eseguendo lo script di laboratorio con privilegi di amministratore. Lo script costruisce la rete (host, switch e router), abilita l'inoltro IP su R1 e installa la tabella di instradamento che collega le due sottoreti. Questo passaggio e' necessario perche' senza il router e le rotte i pacchetti tra la rete 10.0.0.0/24 (dove si trova il client H1) e la rete 172.16.0.0/12 (dove si trova il server H4) non potrebbero attraversare la rete. Al termine compare il prompt mininet>, segno che la rete e' attiva e pronta a ricevere comandi.

```
[analyst@secOps ~]$ sudo lab.support.files/scripts/cyberops_topo.py
```

```
[analyst@secOps ~]$ sudo lab.support.files/scripts/cyberops_topo.py
[sudo] password for analyst:

CyberOPS Topology:

      -----
      | R1 |-----| H4 |
      -----
      |
      |
      -----
|-----| S1 |-----|
|       |       |
|       |       |
|       |       | | | |
|---|---|---|---|---|
| H1 |   | H2 |   | H3 |
|-----|-----|

*** Adding internal links
*** Creating network
*** Adding hosts:
H1 H2 H3 H4 R1
*** Adding switches:
s1
*** Adding links:
(H1, s1) (H2, s1) (H3, s1) (H4, R1) (s1, R1)
*** Configuring hosts
H1 H2 H3 H4 R1 Enabling IP forwarding on R1

*** Starting controller

*** Starting 1 switches
s1 ...
*** Routing Table on Router:
Kernel IP routing table
Destination    Gateway         Genmask         Flags Metric Ref    Use Iface
10.0.0.0        0.0.0.0         255.255.255.0   U        0      0        0 R1-eth1
172.16.0.0      0.0.0.0         255.240.0.0     U        0      0        0 R1-eth2

*** Starting CLI:
mininet> 
```

Figura 1: topologia CyberOps avviata in Mininet, con tabella di routing su R1 (10.0.0.0/24 su R1-eth1 e 172.16.0.0/12 su R1-eth2).

3.2 Avvio del server, del browser e cattura con tcpdump

Dal prompt di Mininet ho aperto i terminali grafici degli host con xterm H1 e xterm H4, poi ho avviato il web server nginx su H4. Su H1, poiche' Firefox non puo' essere eseguito dall'account root per ragioni di sicurezza, sono passato all'utente non privilegiato con su analyst prima di lanciare il browser. Ho quindi avviato la cattura del traffico sull'interfaccia H1-eth0 limitandola a 50 pacchetti, e ho navigato verso 172.16.0.40 con Firefox per generare la sessione TCP da osservare. La cattura serve a fotografare ogni pacchetto che entra ed esce dal client durante la richiesta web: e' su questa fotografia che lavorero' nelle fasi successive. Al termine tcpdump ha confermato 50 pacchetti catturati e 0 scartati dal kernel, a riprova che la cattura e' completa e affidabile.

```
[root@secOps analyst]# /home/analyst/lab.support.files/scripts/reg_server_start.sh
[root@secOps analyst]# su analyst
[analyst@secOps ~]$ firefox &
[analyst@secOps ~]$ sudo tcpdump -i H1-eth0 -v -c 50 -w /home/analyst/capture.pcap
```

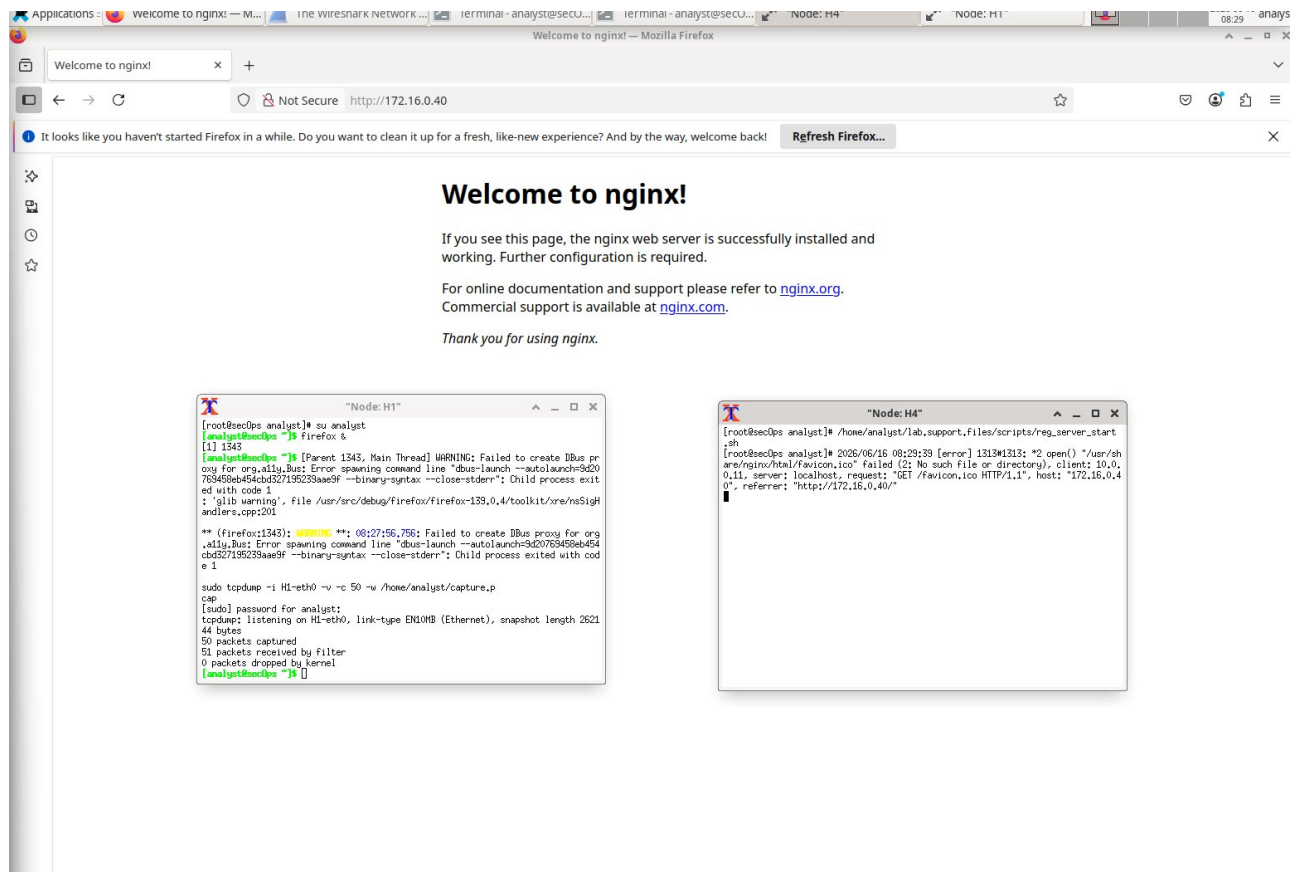


Figura 2: pagina di benvenuto nginx servita da H4 e, nei terminali, l'esito della cattura: 50 packets captured, 51 received by filter, 0 dropped by kernel.

4. Parte 2: Analisi dei Pacchetti con Wireshark

4.1 Avvio di Wireshark e filtro di visualizzazione tcp

Ho aperto Wireshark sull'host H1 e ho caricato il file capture.pcap salvato in precedenza. Per isolare il traffico di interesse ho applicato il filtro di visualizzazione tcp, che nasconde tutti i pacchetti non TCP (ad esempio ARP o DNS) e lascia visibili solo i segmenti della connessione. Il filtro non modifica la cattura: agisce solo sulla visualizzazione, rendendo immediato individuare i tre segmenti dell'handshake. Nella mia cattura l'handshake corrisponde ai frame 34, 35 e 36, appartenenti allo stesso stream TCP (Stream index 0), seguiti al frame 37 dalla prima richiesta applicativa GET / HTTP/1.1.

```
[analyst@secOps ~]$ wireshark-gtk &
```

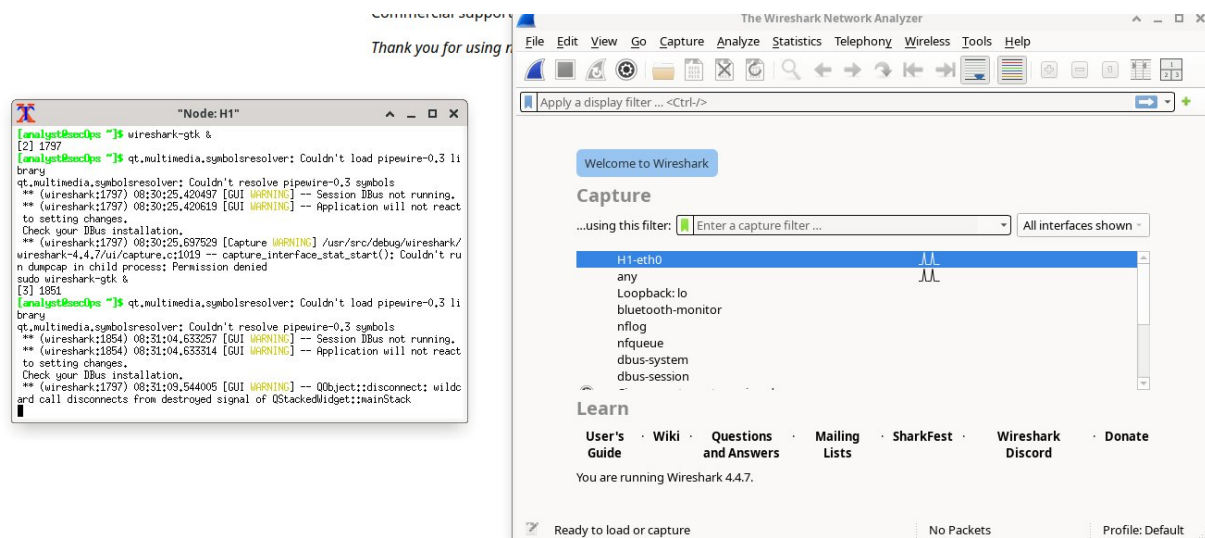


Figura 3: avvio di Wireshark 4.4.7 sull'host H1 dalla finestra di terminale Node: H1.

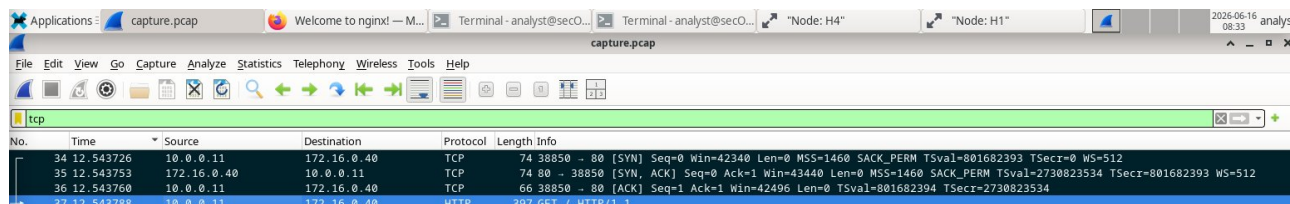


Figura 4: applicato il filtro tcp: i frame 34, 35 e 36 rappresentano i tre segmenti dell'handshake; il frame 37 e' la successiva richiesta HTTP GET /.

4.2 Frame 34: il primo segmento SYN

Il primo segmento parte dal client H1 (10.0.0.11) verso il server H4 (172.16.0.40). E' il pacchetto con cui il client chiede di aprire una connessione: porta il solo flag SYN (0x002) e un numero di sequenza relativo pari a 0, scelto come punto di partenza della numerazione. La porta di destinazione e' la 80, la well-known port del servizio HTTP; la porta di origine e' la 38850, assegnata dinamicamente dal sistema operativo del client. In senso stretto la 38850 ricade nell'intervallo IANA delle porte registrate (1024-49151), ma dal punto di vista funzionale e' una porta effimera, perche' Linux preleva di default le porte sorgente nell'intervallo 32768-60999.

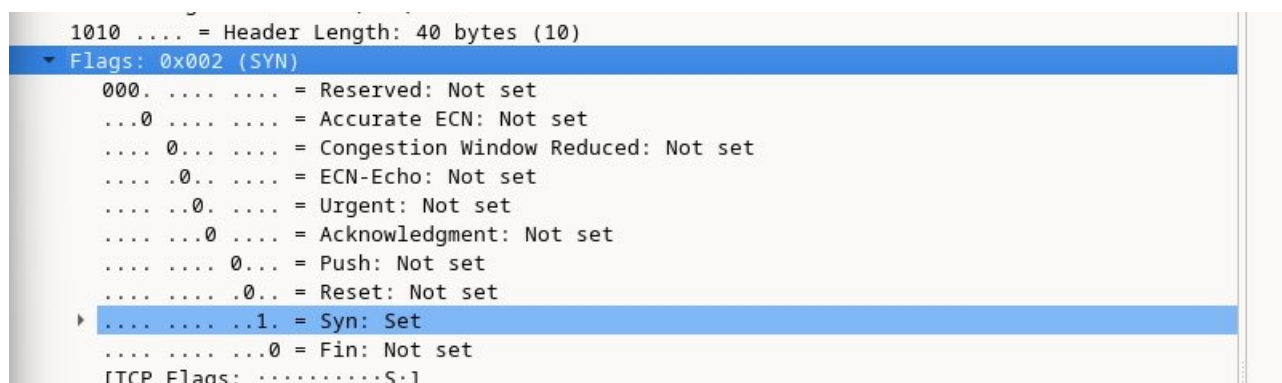


Figura 5: frame 34, dettaglio dei flag TCP: e' impostato il solo bit SYN (Flags 0x002).

Domanda	Risposta
Porta TCP di origine	38850
Classificazione porta di origine	porta effimera/dinamica assegnata dall'OS del client
Porta TCP di destinazione	80
Classificazione porta di destinazione	well-known port (HTTP)
Flag impostato	SYN (0x002)
Sequence number relativo	0

4.3 Frame 35: il segmento SYN-ACK del server

Il secondo segmento e' la risposta del server: viaggia da 172.16.0.40 (porta 80) verso 10.0.0.11 (porta 38850), quindi con porte invertite rispetto al frame precedente. Porta due flag impostati contemporaneamente, SYN e ACK (0x012): il SYN comunica il numero di sequenza iniziale del server (relativo 0), mentre l'ACK conferma la ricezione del SYN del client portando l'acknowledgment relativo a 1, cioe' il numero di sequenza del client incrementato di uno. Wireshark stesso lo segnala con la nota interna che indica come questo sia un ACK al segmento del frame 34. Significa che il server ha accettato la richiesta di connessione e propone a sua volta i propri parametri.

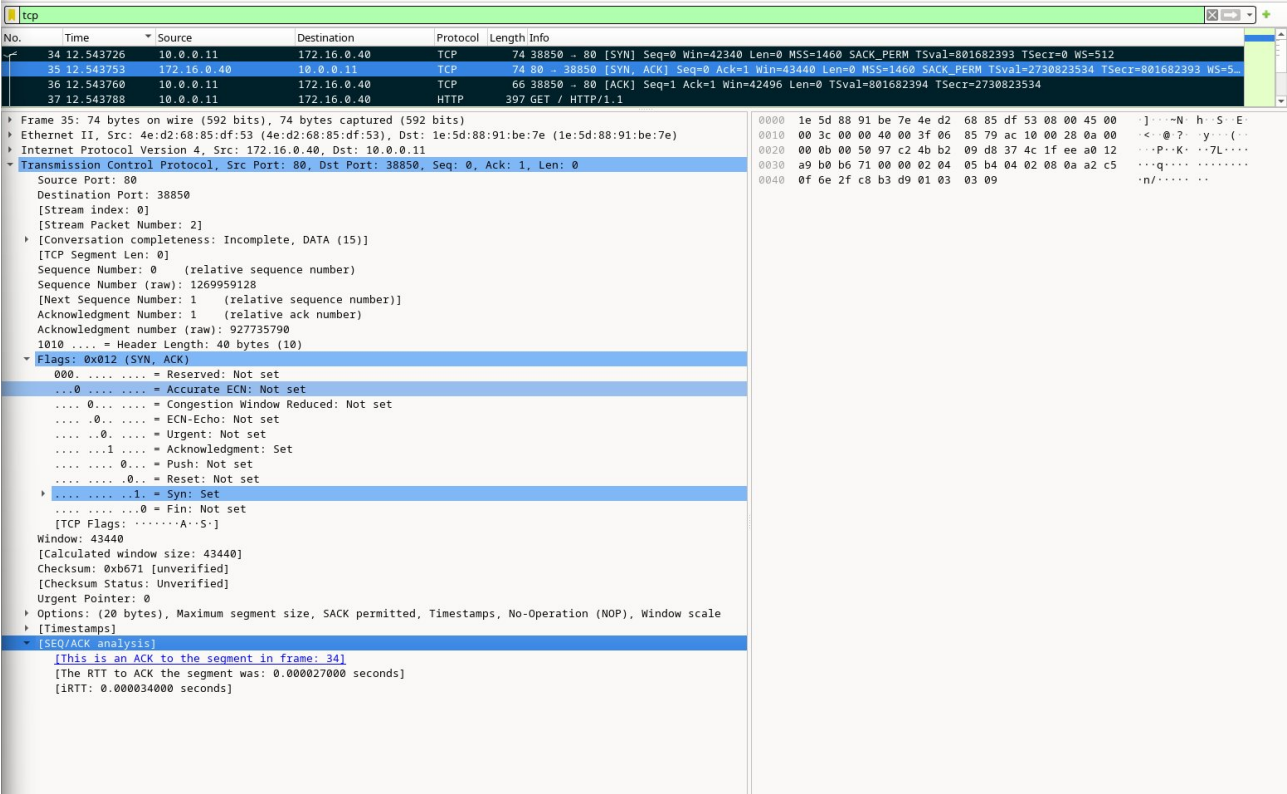


Figura 6: frame 35, segmento SYN-ACK: Flags 0x012, Acknowledgment relativo 1 e analisi SEQ/ACK che lo lega al frame 34.

Domanda	Risposta
Porte (origine - destinazione)	80 e 38850 (invertite: e' la risposta del server)
Flag impostati	SYN, ACK (0x012)
Sequence number relativo	0
Acknowledgment number relativo	1

4.4 Frame 36: il segmento ACK finale

Il terzo e ultimo segmento dell'handshake torna dal client al server e porta il solo flag ACK (0x010). Con questo pacchetto il client conferma il SYN del server: il sequence number relativo passa a 1 e l'acknowledgment relativo e' 1, anch'esso indicato da Wireshark come ACK al segmento del frame 35. A questo punto entrambi gli host hanno confermato i rispettivi numeri di sequenza iniziali: la connessione TCP e' stabilita e la comunicazione applicativa puo' iniziare, come si vede nel frame 37 con la richiesta HTTP GET /.

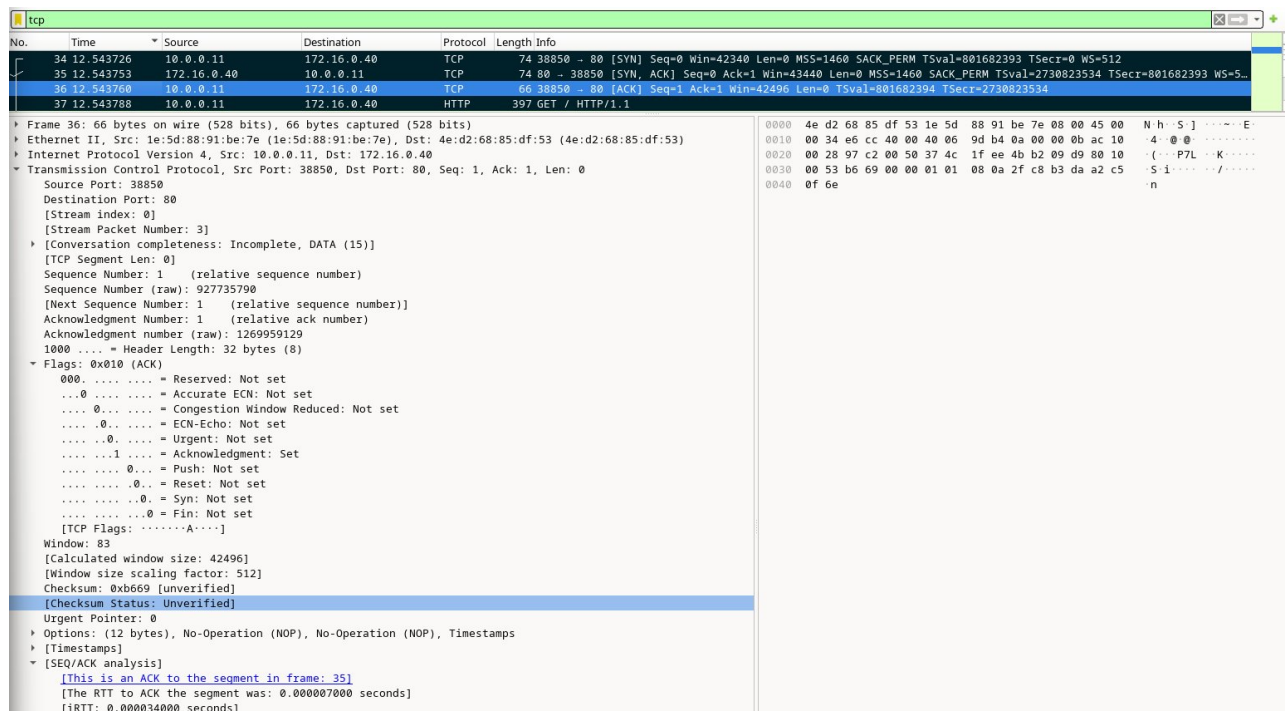


Figura 7: frame 36, segmento ACK finale: Flags 0x010, sequence relativo 1 e acknowledgment relativo 1; l'header TCP scende a 32 byte.

Domanda	Risposta
Flag impostato	ACK (0x010)
Sequence number relativo	1
Acknowledgment number relativo	1

4.5 Tabella riepilogativa dell'handshake

La tabella seguente riassume i tre segmenti e rende evidente la logica della negoziazione: il SYN apre, il SYN-ACK risponde e conferma, l'ACK chiude la fase di instaurazione. I numeri di sequenza e acknowledgment relativi partono entrambi da valori bassi (0 e 1) proprio perche' Wireshark li mostra in forma relativa al punto di partenza, per facilitarne la lettura.

Frame	Direzione (IP:porta)	Flag	Seq rel	Ack rel
34	10.0.0.11:38850 > 172.16.0.40:80	SYN (0x002)	0	-
35	172.16.0.40:80 > 10.0.0.11:38850	SYN, ACK (0x012)	0	1
36	10.0.0.11:38850 > 172.16.0.40:80	ACK (0x010)	1	1

5. Parte 3: Visualizzazione dei Pacchetti con tcpdump

5.1 Consultazione del manuale e opzione -r

Prima di rileggere la cattura ho consultato il manuale di tcpdump con `man tcpdump`, per individuare l'opzione adatta a lavorare su un file gia' salvato. Nelle man page e' possibile cercare un termine digitando / seguito dalla stringa (ad esempio `-r`), passare alla corrispondenza successiva con `n` e uscire con `q`. Consultare il manuale e' una buona pratica perche' permette di scegliere le opzioni corrette senza affidarsi alla memoria.

Cosa fa l'opzione `-r`: indica a tcpdump di leggere i pacchetti da un file di cattura salvato (un file .pcap) invece di catturarli in tempo reale dall'interfaccia di rete. In pratica abilita l'analisi offline di una cattura

prodotta in precedenza.

```

NAME
    tcpdump - dump traffic on a network

SYNOPSIS
    tcpdump [ -AbDefhHIJKlLnOpqStuUvxx# ] [ -B buffer_size ]
    [ -c count ] [ --count ] [ -C file_size ]
    [ -E spi@ipaddr algo:secret,... ]
    [ -F file ] [ -G rotate_seconds ] [ -i interface ]
    [ --immediate-mode ] [ -j tstamp_type ] [ -m module ]
    [ -M secret ] [ --number ] [ --print ] [ -Q in|out|inout ]
    [ -r file ] [ -s snaplen ] [ -T type ] [ --version ]
    [ -V file ] [ -w file ] [ -W filecount ] [ -y datalinktype ]
    [ -z postrotate-command ] [ -Z user ]
    [ --time-stamp-precision=tstamp_precision ]
    [ --micro ] [ --nano ]
    [ expression ]

DESCRIPTION
    Tcpdump prints out a description of the contents of packets on a network
    interface that match the Boolean expression (see pcap-filter(7) for the
    expression syntax); the description is preceded by a time stamp,
    printed, by default, as hours, minutes, seconds, and fractions of a sec-
    ond since midnight. It can also be run with the -w flag, which causes
    it to save the packet data to a file for later analysis, and/or with the
    -r flag, which causes it to read from a saved packet file rather than to
    read packets from a network interface. It can also be run with the -V
    flag, which causes it to read a list of saved packet files. In all
    cases, only packets that match expression will be processed by tcpdump.

    Tcpdump will, if not run with the -c flag, continue capturing packets
    until it is interrupted by a SIGINT signal (generated, for example, by
    typing your interrupt character, typically control-C) or a SIGTERM sig-
    nal (typically generated with the kill(1) command); if run with the -c
    flag, it will capture packets until it is interrupted by a SIGINT or
    SIGTERM signal or the specified number of packets have been processed.

    When tcpdump finishes capturing packets, it will report counts of:

        packets 'captured' (this is the number of packets that tcpdump
        has received and processed);

        packets 'received by filter' (the meaning of this depends on
        the OS on which you're running tcpdump, and possibly on the way
        the OS was configured - if a filter was specified on the command
        line, on some OSes it counts packets regardless of whether they
        were matched by the filter expression and, even if they were
        matched by the filter expression, regardless of whether tcpdump
        has read and processed them yet, on other OSes it counts only
        packets that were matched by the filter expression regardless of
        whether tcpdump has read and processed them yet, and on other
        OSes it counts only packets that were matched by the filter ex-
        pression and were processed by tcpdump);

Manual page tcpdump(1) line 1 (press h for help or q to quit)

```

Figura 8: pagina di manuale di tcpdump: nella sezione SYNOPSIS e' visibile l'opzione -r file per la lettura da file.

5.2 Lettura offline della cattura

Ho riletto il file capture.pcap filtrando il solo traffico TCP e limitando l'output ai primi pacchetti. Ho usato -c 4 invece di -c 3 per includere, oltre ai tre segmenti dell'handshake, anche la prima richiesta applicativa: la traccia stessa suggerisce di aumentare il numero di righe dopo -c quando si vuole vedere l'handshake completo seguito dai dati. Nell'output ritrovo la stessa sequenza osservata in Wireshark: il segmento con flag [S] (SYN) dal client, il segmento [S.] (SYN-ACK) dal server, il segmento [.] (ACK) dal client e infine il segmento [P.] con la richiesta HTTP GET /. La notazione compatta di tcpdump conferma quindi la lettura grafica fatta in precedenza.

```
[analyst@secOps ~]$ tcpdump -r /home/analyst/capture.pcap tcp -c 4
```

```
[analyst@secOps ~]$ tcpdump -r /home/analyst/capture.pcap tcp -c 4
reading from file /home/analyst/capture.pcap, link-type EN10MB (Ethernet), snapshot length 262144
08:29:39.684104 IP 10.0.0.11.38850 > 172.16.0.40.http: Flags [S], seq 927735789, win 42340, options [mss 1460,sackOK,TS val 801682393 ecr 0,nop,wscale 9], length 0
08:29:39.684131 IP 172.16.0.40.http > 10.0.0.11.38850: Flags [S.], seq 1269959128, ack 927735790, win 43440, options [mss 1460,sackOK,TS val 2730823534 ecr 801682393,nop,wscale 9], length 0
08:29:39.684138 IP 10.0.0.11.38850 > 172.16.0.40.http: Flags [.], ack 1, win 83, options [nop,nop,TS val 801682394 ecr 2730823534], length 0
08:29:39.684166 IP 10.0.0.11.38850 > 172.16.0.40.http: Flags [P.], seq 1:332, ack 1, win 83, options [nop,nop,TS val 801682394 ecr 2730823534], length 331: HTTP: GET / HTTP/1.1
[analyst@secOps ~]$
```

Figura 9: lettura offline con tcpdump -r: si riconoscono i flag [S], [S.], [.], e infine [P.] con la richiesta GET / HTTP/1.1.

6. Chiusura dell'Ambiente

Terminata l'analisi ho chiuso Mininet digitando quit nella finestra principale: la procedura arresta switch, host e link e disabilita l'inoltro IP su R1. Per ripulire eventuali processi e interfacce rimasti attivi ho infine eseguito `sudo mn -c`, che rimuove controller, datapath OVS, tunnel e processi residui. Questa pulizia e' importante perche' evita che configurazioni fantasma interferiscano con i laboratori successivi.

```
mininet> quit
[analyst@secOps ~]$ sudo mn -c
```

```
*** Starting CLI:
mininet> xterm H1
mininet> xterm H4
mininet> xterm H1
mininet> quit
*** Stopping 0 controllers

*** Stopping 3 terms
*** Stopping 5 links
. . . . .
*** Stopping 1 switches
s1
*** Stopping 5 hosts
H1 H2 H3 H4 R1 Disabling IP forwarding on R1

*** Done
[analyst@secOps ~]$
```

Figura 10: arresto di Mininet con quit: stopping di controller, link, switch e host.

```
[analyst@secOps ~]$ sudo mn -c
[sudo] password for analyst:
*** Removing excess controllers/ofprotocols/ofdatapaths/pings/noxes
killall controller ofprotocol ofdatapath ping nox_core lt-nox_core ovs-openflowd ovs-controller ovs-testcontroller udpbwtest mnexec ivs ryu-manager 2> /dev/null
killall -9 controller ofprotocol ofdatapath ping nox_core lt-nox_core ovs-openflowd ovs-controller ovs-testcontroller udpbwtest mnexec ivs ryu-manager 2> /dev/null
pkill -9 -f "sudo mnexec"
*** Removing junk from /tmp
rm -f /tmp/vconn* /tmp/vlogs* /tmp/*.out /tmp/*.log
*** Removing old X11 tunnels
*** Removing excess kernel datapaths
ps ax | egrep -o 'dp[0-9]+' | sed 's/dp/nl:/'
egrep: warning: egrep is obsolescent; using grep -E
*** Removing OVS datapaths
ovs-vsctl --timeout=1 list-br
ovs-vsctl --timeout=1 list-br
*** Removing all links of the pattern foo-ethx
ip link show | egrep -o '([_[:alnum:]]+-eth[[:digit:]]+)'
egrep: warning: egrep is obsolescent; using grep -E
ip link show
*** Killing stale mininet node processes
pkill -9 -f mininet:
*** Shutting down stale tunnels
pkill -9 -f Tunnel-Ethernet
pkill -9 -f .ssh/mn
rm -f ~/.ssh/mn/*
*** Cleanup complete.
```

Figura 11: pulizia dei processi residui con sudo mn -c, fino al messaggio Cleanup complete.

7. Riepilogo dei Comandi

Comando	Descrizione
<code>sudo lab.support.files/scripts/cyberops_topo.py</code>	avvia la topologia Mininet
<code>xterm H1 / xterm H4</code>	apre i terminali grafici degli host
<code>reg_server_start.sh</code>	avvia il web server nginx su H4
<code>su analyst</code>	passa da root all'utente analyst su H1
<code>firefox &</code>	avvia il browser su H1
<code>sudo tcpdump -i H1-eth0 -v -c 50 -w ../capture.pcap</code>	cattura 50 pacchetti su file
<code>wireshark-gtk &</code>	avvia Wireshark
<code>tcp</code>	filtro di visualizzazione per il solo traffico TCP
<code>man tcpdump</code>	consulta il manuale di tcpdump
<code>tcpdump -r ../capture.pcap tcp -c 4</code>	rilegge offline i primi pacchetti TCP
<code>quit</code>	chiude Mininet
<code>sudo mn -c</code>	pulisce i processi e le interfacce residue

8. Domande di Riflessione

1. Tre filtri utili a un amministratore di rete.

`http`: isola il traffico web per verificare richieste e risposte applicative.

`dns`: controlla le risoluzioni dei nomi e individua comportamenti anomali, come possibili tunneling o esfiltrazione tramite DNS.

`ip.addr == 10.0.0.11` oppure `tcp.port == 80`: concentra l'analisi su un host o un servizio specifico. Si possono aggiungere filtri come `tcp.flags.syn == 1 && tcp.flags.ack == 0` per evidenziare tentativi di scansione delle porte.

2. Altri usi di Wireshark in una rete di produzione.

Oltre alla didattica, Wireshark è impiegato per il troubleshooting di problemi di rete e latenza, per l'analisi di sicurezza e l'incident response (individuazione di traffico anomalo, malware o esfiltrazione di dati), per la creazione di baseline del traffico utili al capacity planning, per il debug di applicazioni a livello di protocollo e per la verifica di configurazioni come l'handshake TLS o la presenza di ritrasmissioni.

9. Conclusioni

Ho catturato una sessione TCP reale e ne ho analizzato l'apertura sia graficamente con Wireshark sia testualmente con tcpdump, ottenendo lo stesso risultato con due strumenti diversi. L'esercizio mi ha permesso di osservare in modo concreto la sequenza SYN, SYN-ACK, ACK e di capire come i flag, le porte e i numeri di sequenza e acknowledgment cooperino per stabilire una connessione affidabile. La capacità di filtrare il traffico e di rileggere una cattura offline è una competenza di base sia per chi amministra una rete sia per chi svolge analisi di sicurezza, perché permette di passare rapidamente da una massa di pacchetti all'esatta porzione di traffico rilevante.